Authentication vs. Authorization

The Fundamental Boundary: Who Are You? vs. What May You Do?

AUTHENTICATION (AuthN)

"WHO ARE YOU?" — IDENTITY VERIFICATION

Proof of Claim:
Validates proof: password hashes (Argon2id), WebAuthn cryptographic keys, or asymmetric signatures.

MFA Challenges:
Combines independent factors: knowledge (password), possession (FIDO2 token, TOTP), or biometrics.

Output Artifact:
Establishes a verified Principal (e.g. sub: "usr_alice99").

AUTHORIZATION (AuthZ)

"WHAT MAY YOU DO?" — ACCESS PERMISSION

Policy Evaluation:
Evaluates whether the verified principal holds permissions for the targeted operation on a specific object.

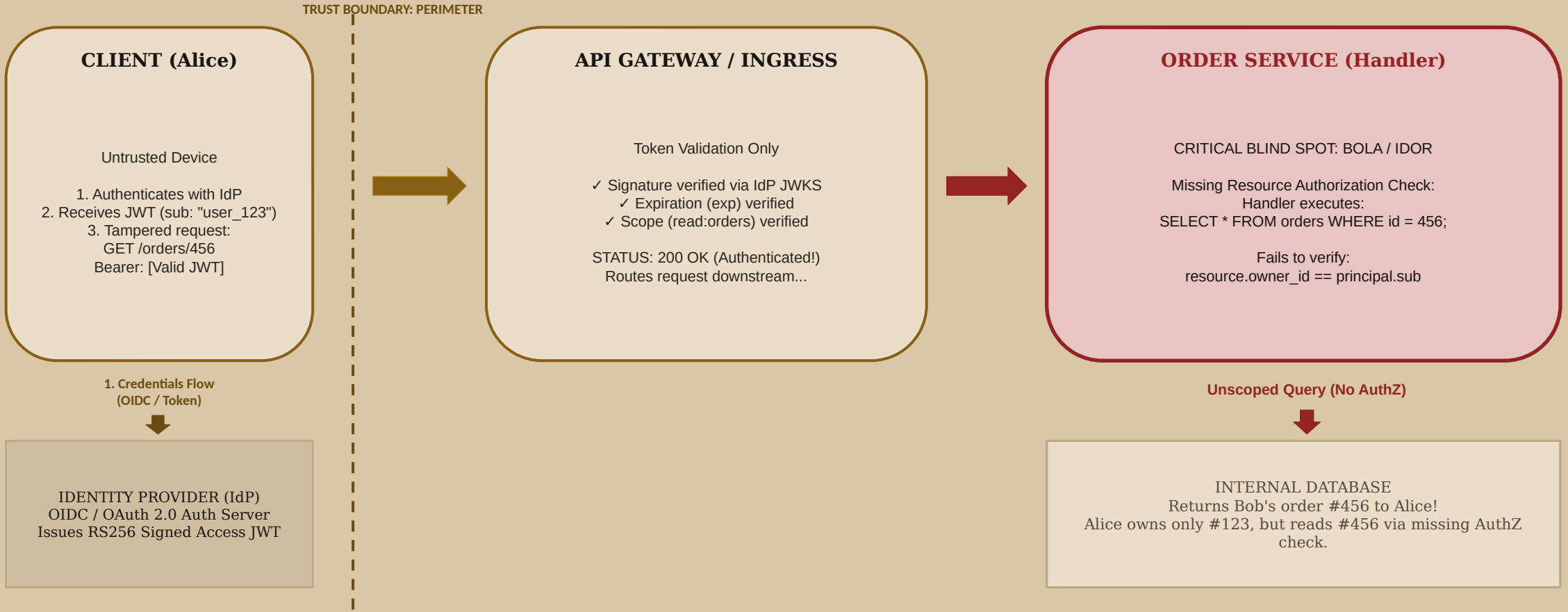
Contextual Governance:
Incorporates resource ownership, enterprise tenancy, environmental attributes, and relationships.

Output Decision:
Deterministic Allow or Deny (HTTP 200 OK or 403 Forbidden).

ARCHITECTURAL RULE: Authentication establishes identity; it NEVER confers entitlement. A valid 200 OK login does not grant resource access.

Valid Identity, Wrong Resource

The Missing Ownership Check & Broken Object-Level Authorization (BOLA)



How SQL Injection Changes a Query

When Untrusted Input Mutates the Database Parser AST

VULNERABLE QUERY & ATTACK PAYLOAD

```
// 1. Flawed Application Code (String Concatenation)
const query = "SELECT * FROM users WHERE email = '" + userInput + "' AND
active = 1;";

// 2. Fictional Attacker Payload
userInput = "admin@corp.internal' OR 1=1 --"

// 3. Executed Query at Database Engine
SELECT * FROM users WHERE email = 'admin@corp.internal' OR 1=1 --' AND
active = 1;
```

HOW THE PARSER IS HIJACKED

1. Literal Boundary Escape (')

The injected apostrophe terminates the string literal token. The database lexer exits data mode and begins parsing raw SQL tokens.

2. AST Logic Mutation (OR 1=1)

The parser creates a new binary operator node in the Abstract Syntax Tree. The tautology (1=1) forces the WHERE clause to evaluate TRUE for all records.

3. Grammar Truncation (--)

The SQL comment delimiter forces the parser to discard the rest of the query, stripping developer tenant & active-state constraints.

Defense in Depth: Code vs. Data

Structural Invariance & Multi-Layered Security Controls

PRIMARY SHIELD: PARAMETERIZATION (STRUCTURAL INVARIANCE)

```
Phase 1: Parse Skeleton [Fixed Immutable AST]
SELECT id, name FROM users WHERE email = $1;
```

```
Phase 2: Separate Wire Bind Packet (Slot $1)
Input "admin' OR 1=1 --" is treated strictly as literal data bytes!
```

```
// Production Driver Pattern: await db.query("SELECT id, name FROM users WHERE email = $1 AND active = $2;", [userInput, true]);
```

1. Least-Privilege DB Roles

Purpose: DML grants only (SELECT, INSERT, UPDATE). Blocks DDL (DROP, ALTER) & OS reads.
Limit: Cannot stop theft from granted tables. Blast-radius containment only.

2. WAF & Schema Types

Purpose: Strict schema contracts (Zod/UUIDs) reject edge malformed payloads.
Limit: Freeform text requires quotes. Validation aids hygiene, not parameterization.

3. Safe ORMs & CI Linters

Purpose: ORMs parameterize ASTs automatically by default.
Limit: Escape hatches ($queryRawUnsafe) reintroduce injection. CI linters must ban them.

The Engineering Rule

A Unified Mental Model for Production Systems

“Verify identity. Authorize the resource. Keep data out of code.”

THE UNIFIED ARCHITECTURAL TRIAD FOR PRODUCTION ENGINEERING

1. Centralized AuthZ Middleware

Decouple policy from route handlers. Enforce resource ownership checks (e.g. OPA / Cedar / ReBAC) on every internal endpoint.

2. Static AST Guardrails in CI

Block pull requests automatically using Semgrep & ESLint rules that flag string interpolation in query runners before code reaches production.

3. Integration Boundary Tests

Mandate automated test suites that explicitly pass cross-tenant IDs and verify 403 Forbidden responses, proving isolation works under adversarial test cases.